

COURSE CODE/TITLE:	CPE 212	SECTION:	CPE31S2
DATE PERFORMED:	Aug 27, 2026	DATE SUBMITTED:	Aug 29, 2026
NAME:	Loterte, Mark David M.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 5	Implement Ansible roles in playbooks
-----------------------	---

1. PROCEDURE OUTPUT

Task 1: Use when command for different distributions

1. In the local machine, make sure you are in the local repository directory (CPE232_yourname). Issue the command git pull. When prompted, enter the correct passphrase or password. Describe what happened when you issue this command. Did something happen? Why?

```
mdmlot@workstation:~/ansible-lab$ ls ../CPE212_loterte
install_apache.yml  README.md
mdmlot@workstation:~/ansible-lab$
```

No text appeared on the command line interface, indicating a successful pull because I first did HOA 4 at home and then pushed the playbook to github. Pulling again results the following.

```
mdmlot@workstation:~/CPE212_loterte$ git pull
Already up to date.
```

2. Edit the inventory file and add the IP address of the Centos VM. Issue the command we used to execute the playbook (the one we used in the last activity): `ansible-playbook --ask-become-pass install_apache.yml`. After executing this command, you may notice that it did not become successful in the Centos VM. You can see that the Centos VM has failed=1. Only the two remote servers have been changed. The reason is that Centos VM does not support "apt" as the package manager. The default package manager for Centos is "yum."

```
mdm1ot@workstation:~/ansible-lab$ ansible-playbook --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.110]
ok: [192.168.56.113]
ok: [192.168.56.114]

TASK [Update Repository Index] *****
[WARNING]: Updating cache and auto-installing missing dependency: python3-apt
fatal: [192.168.56.114]: FAILED! => {"changed": false, "cmd": "apt-get update", "msg": "[Errno 2] No s
ch file or directory: b'apt-get'", "rc": 2}
changed: [192.168.56.110]
changed: [192.168.56.113]

TASK [Apache Install] *****
changed: [192.168.56.110]
changed: [192.168.56.113]

TASK [Add PHP support for Apache] *****
ls
changed: [192.168.56.113]
changed: [192.168.56.110]

PLAY RECAP *****
192.168.56.110      : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
192.168.56.113      : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
192.168.56.114      : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0
  ignored=0
```

Edit the install_apache.yml file and insert the lines shown below.

Make sure to save the file and exit.

Run ansible-playbook --ask-become-pass install_apache.yml and describe the result.

```

ndm1ot@workstation:~/CPE212_loterte$ ansible-playbook -i inventory.yaml --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [server2]
ok: [server1]
ok: [server3]

TASK [Update Repository Index] *****
skipping: [server3]
changed: [server2]
changed: [server1]

TASK [Apache Install] *****
skipping: [server3]
ok: [server1]
ok: [server2]

TASK [Add PHP support for Apache] *****
skipping: [server3]
ok: [server1]
ok: [server2]

PLAY RECAP *****
server1      : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
server2      : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
server3      : ok=1    changed=0    unreachable=0    failed=0    skipped=3    rescued=0
  ignored=0

```

Using the when statement, the commands in the playbook only ran on the Ubuntu systems, hence the lesser number of run commands via ok=1 in server3 compared to ok=4 in server2.

If you have a mix of Debian and Ubuntu servers, you can change the configuration of your playbook like this.

- name: update repository index

apt:

update_cache: yes

when: ansible_distribution in ["Debian", "Ubuntu"]

Note: This will work also if you try. Notice the changes are highlighted.

```
mdm1ot@workstation:~/CPE212_loterte$ ansible-playbook -i inventory.yaml --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [server2]
ok: [server1]
ok: [server3]

TASK [Update Repository Index] *****
skipping: [server3]
changed: [server2]
changed: [server1]

TASK [Apache Install] *****
skipping: [server3]
ok: [server1]
ok: [server2]

TASK [Add PHP support for Apache] *****
skipping: [server3]
ok: [server1]
ok: [server2]

PLAY RECAP *****
server1      : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
server2      : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
server3      : ok=1    changed=0    unreachable=0    failed=0    skipped=3    rescued=0
  ignored=0

mdm1ot@workstation:~/CPE212_loterte$
```

The output is the same. However, the playbook this time is accounting for Debian Operating Systems.

Edit the install_apache.yml file and insert the lines shown below.

Make sure to save and exit.

Run `ansible-playbook --ask-become-pass install_apache.yml` and describe the result.

```
TASK [Update Repository Index] *****
skipping: [server3]
changed: [server1]
changed: [server2]

TASK [Apache Install] *****
skipping: [server3]
ok: [server1]
ok: [server2]

TASK [Add PHP support for Apache] *****
skipping: [server3]
ok: [server2]
ok: [server1]

TASK [Update Repository Index] *****
skipping: [server1]
skipping: [server2]
ok: [server3]

TASK [Apache Install] *****
skipping: [server1]
skipping: [server2]
changed: [server3]

TASK [Add PHP support for Apache] *****
skipping: [server1]
skipping: [server2]
changed: [server3]

PLAY RECAP *****
server1      : ok=4    changed=1    unreachable=0    failed=0    skipped=3    rescued=0
              ignored=0
server2      : ok=4    changed=1    unreachable=0    failed=0    skipped=3    rescued=0
              ignored=0
server3      : ok=4    changed=2    unreachable=0    failed=0    skipped=3    rescued=0
              ignored=0
```

Accounting for CentOS installations required creating new tasks that only looked for CentOS installations which ignored the Ubuntu installations.

To verify the installations, go to CentOS VM and type its IP address on the browser. Was it successful? The answer is no. It's because the httpd service or the Apache HTTP server in the CentOS is not yet active. Thus, you need to activate it first.

To activate, go to the CentOS VM terminal and enter the following:
systemctl status httpd
The result of this command tells you that the service is inactive.

```
mdmlot@server3:~ — systemctl status httpd
[mdmlot@server3 ~]$ systemctl status httpd
o httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: d>
   Drop-In: /usr/lib/systemd/system/httpd.service.d
            └─php-fpm.conf
   Active: inactive (dead)
   Docs: man:httpd.service(8)
lines 1-6/6 (END)
```

Issue the following command to start the service:

```
sudo systemctl start httpd
```

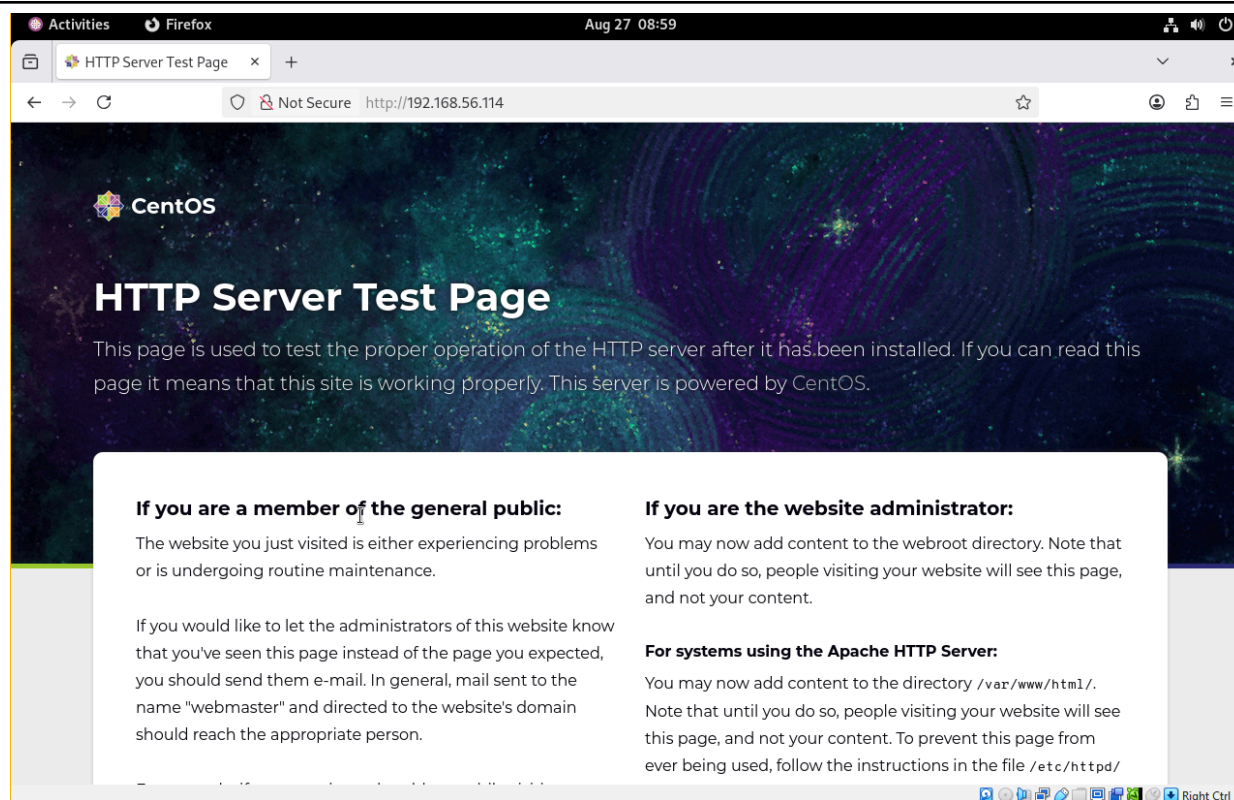
(When prompted, enter the sudo password)

```
sudo firewall-cmd --add-port=80/tcp
```

(The result should be a success)

```
[mdmlot@server3 ~]$ sudo systemctl start httpd
[sudo] password for mdmlot:
[mdmlot@server3 ~]$ sudo firewall-cmd --add-port=80/tcp
success
[mdmlot@server3 ~]$
```

To verify the service is already running, go to CentOS VM and type its IP address on the browser. Was it successful? (Screenshot the browser)



Task 2: Refactoring playbook

This time, we want to make sure that our playbook is efficient and that the codes are easier to read. This will also makes run ansible more quickly if it has to execute fewer tasks to do the same thing.

Edit the playbook `install_apache.yml`. Currently, we have three tasks targeting our Ubuntu machines and 3 tasks targeting our CentOS machine. Right now, we try to consolidate some tasks that are typically the same. For example, we can consolidate two plays that install packages. We can do that by creating a list of installation packages as shown below:

Make sure to save the file and exit.

Run `ansible-playbook --ask-become-pass install_apache.yml` and describe the result.

```
e.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [server1]
ok: [server2]
ok: [server3]

TASK [Update Repository Index] *****
skipping: [server3]
changed: [server2]
changed: [server1]

TASK [install apache and php for Ubuntu] *****
skipping: [server3]
ok: [server2]
ok: [server1]

TASK [Update Repository Index] *****
skipping: [server1]
skipping: [server2]
ok: [server3]

TASK [install apache and php for CentOS] *****
skipping: [server1]
skipping: [server2]
ok: [server3]

PLAY RECAP *****
server1      : ok=3    changed=1    unreachable=0    failed=0    skipped=2    rescued=0
  ignored=0
server2      : ok=3    changed=1    unreachable=0    failed=0    skipped=2    rescued=0
  ignored=0
server3      : ok=3    changed=0    unreachable=0    failed=0    skipped=2    rescued=0
  ignored=0
```

The output is still successful but the counters of ok in each server was reduced by 1.

Edit the playbook install_apache.yml again. In task 2.1, we consolidated the plays into one play. This time we can actually consolidated everything in just 2 plays. This can be done by removing the update repository play and putting the command update_cache: yes below the command state: latest. See below for reference:

Make sure to save the file and exit.

Run ansible-playbook --ask-become-pass install_apache.yml and describe the result.

```
PLAY RECAP *****
server1      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0
  ignored=0
server2      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0
  ignored=0
server3      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0
  ignored=0

mdm1ot@workstation:~/CPE212_loterte$ ansible-playbook -i inventory.yaml --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [server2]
ok: [server1]
ok: [server3]

TASK [install apache and php for Ubuntu] *****
skipping: [server3]
ok: [server1]
ok: [server2]

TASK [install apache and php for CentOS] *****
skipping: [server1]
skipping: [server2]
ok: [server3]

PLAY RECAP *****
server1      : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0
  ignored=0
server2      : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0
  ignored=0
server3      : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0
  ignored=0

mdm1ot@workstation:~/CPE212_loterte$
```

The output was still successful but the number of tasks were further reduced by 1 on each server.

Finally, we can consolidate these 2 plays in just 1 play. This can be done by declaring variables that will represent the packages that we want to install. Basically, the `apache_package` and `php_package` are variables. The names are arbitrary, which means we can choose different names. We also take out the line `when: ansible_distribution`. Edit the playbook `install_apache.yml` again and make sure to follow the below image. Make sure to save the file and exit.

Run `ansible-playbook --ask-become-pass install_apache.yml` and describe the result.

```
mdmlot@workstation:~/CPE212_loterte$ ansible-playbook -i inventory.yaml --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [server2]
ok: [server1]
ok: [server3]

TASK [install apache and php] *****
fatal: [server1]: FAILED! => {"msg": "The task includes an option with an undefined variable. The error was: 'apache_package' is undefined\n\nThe error appears to be in '/home/mdmlot/CPE212_loterte/install_apache.yml': line 6, column 7, but may\nbe elsewhere in the file depending on the exact syntax problem.\n\nThe offending line appears to be:\n\n    - name: install apache and php\n      ^ here\n"}
fatal: [server2]: FAILED! => {"msg": "The task includes an option with an undefined variable. The error was: 'apache_package' is undefined\n\nThe error appears to be in '/home/mdmlot/CPE212_loterte/install_apache.yml': line 6, column 7, but may\nbe elsewhere in the file depending on the exact syntax problem.\n\nThe offending line appears to be:\n\n    - name: install apache and php\n      ^ here\n"}
fatal: [server3]: FAILED! => {"msg": "The task includes an option with an undefined variable. The error was: 'apache_package' is undefined\n\nThe error appears to be in '/home/mdmlot/CPE212_loterte/install_apache.yml': line 6, column 7, but may\nbe elsewhere in the file depending on the exact syntax problem.\n\nThe offending line appears to be:\n\n    - name: install apache and php\n      ^ here\n"}

PLAY RECAP *****
server1                : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0
ignored=0
server2                : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0
ignored=0
server3                : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0
ignored=0

mdmlot@workstation:~/CPE212_loterte$
```

Unfortunately, task 2.3 was not successful. It's because we need to change something in the inventory file so that the variables we declared will be in place.

Edit the inventory file and follow the below configuration:

Make sure to save the inventory file and exit.

```
mdmlot@workstation:~/CPE212_loterte$ ansible-playbook --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.113]
ok: [192.168.56.110]
ok: [192.168.56.114]

TASK [install apache and php] *****
[WARNING]: Updating cache and auto-installing missing dependency: python3-apt
fatal: [192.168.56.114]: FAILED! => {"changed": false, "cmd": "apt-get update", "msg": "[Errno 2] No such file or directory: b'apt-get'", "rc": 2}
ok: [192.168.56.110]
ok: [192.168.56.113]

PLAY RECAP *****
192.168.56.110         : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
192.168.56.113         : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
192.168.56.114         : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0
ignored=0

mdmlot@workstation:~/CPE212_loterte$
```

Finally, we still have one more thing to change in our `install_apache.yml` file. In task 2.3, you may notice that the package is assign as `apt`, which will not run in CentOS. Replace the `apt` with `package`. `Package` is a module in ansible that is generic, which is going to use whatever package manager the underlying host or the target server uses. For Ubuntu it will automatically use `apt`, and for CentOS it will automatically use `dnf`. Make sure to save the file and exit. For more details about the ansible package, you may refer to this documentation:

ansible.builtin.package – Generic OS package manager — Ansible Documentation

Run `ansible-playbook --ask-become-pass install_apache.yml` and describe the result.

```
mdmlot@workstation:~/CPE212_loterte$ ansible-playbook --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.113]
ok: [192.168.56.110]
ok: [192.168.56.114]

TASK [install apache and php] *****
ok: [192.168.56.113]
ok: [192.168.56.110]
ok: [192.168.56.114]

PLAY RECAP *****
192.168.56.110      : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
192.168.56.113      : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
192.168.56.114      : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
mdmlot@workstation:~/CPE212_loterte$
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Create a playbook that could do the previous tasks in Red Hat OS.

```
GNU nano 5.6.1
all:
  vars:
    ansible_user: mdmlot
  hosts:
    workstation:
      ansible_host: 192.168.56.105
      os_type: Ubuntu
      apache_package: apache2
      php_package: libapache2-mod-php
    server1:
      ansible_host: 192.168.56.104
      os_type: Ubuntu
      apache_package: apache2
      php_package: libapache2-mod-php
```

```
--
hosts: all
become: true
tasks:
  - name: Install Apache and PHP
    ansible.builtin.apt:
      name:
        - "{{ apache_package }}"
        - "{{ php_package }}"
      state: latest
      update_cache: yes
```

```
[mdmlot@server2 ~]$ ansible-playbook --ask-become-pass -i inventory.yaml install_apache.yaml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [workstation]
ok: [server1]

TASK [Install Apache and PHP] *****
ok: [server1]
```

Reflections:

Answer the following:

1. Why do you think refactoring of playbook codes is important?

Refactoring Ansible playbooks allows for better maintainability and readability while also allowing for efficiency in running tasks. Stacking layers of configurations and commands in a playbook slows down execution times and

refactoring techniques such as using the when: keyword for only specific operating systems help mitigate this.

2. When do we use the “when” command in playbook?

The when command should be used when dealing with dynamic or different variables across multiple servers such as the type of OS.

Conclusion:

I conclude that Ansible playbooks can be optimized further across server environments by using variables, conditionals, and generic modules. Practicing these refactoring techniques will allow me to create more maintainable and scalable infrastructure by reducing task redundancy, especially across different operating system distros.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.